Concrete Architecture of OpenCV

October 29, 2025

SourceArchitects Group Members:

- Sheida Khodabakhshian Khansari
- Vrinda Ashwar
- Matthew Lee
- Werner Bagu-Paiz
- William Moran
- Stefewn Johnson

Introduction to the Concrete Architecture of OpenCV

- Objective: Extract concrete architecture using SciTools Understand and visualize using LSEdit
- Top-Level Subsystem: Image Processing Subsystem

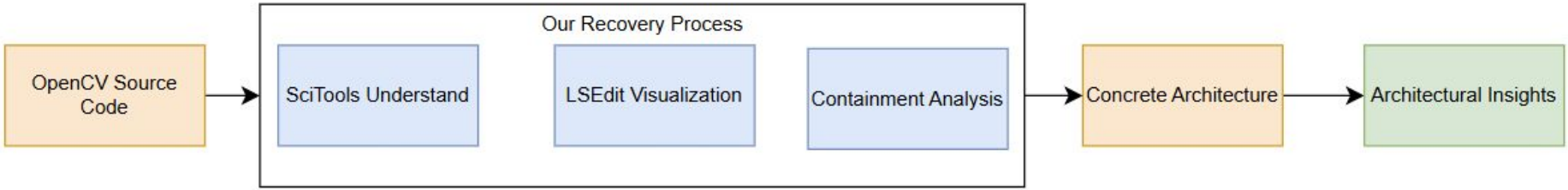


Figure 1: Broad Project Steps Outline

High-Level Concrete Architecture

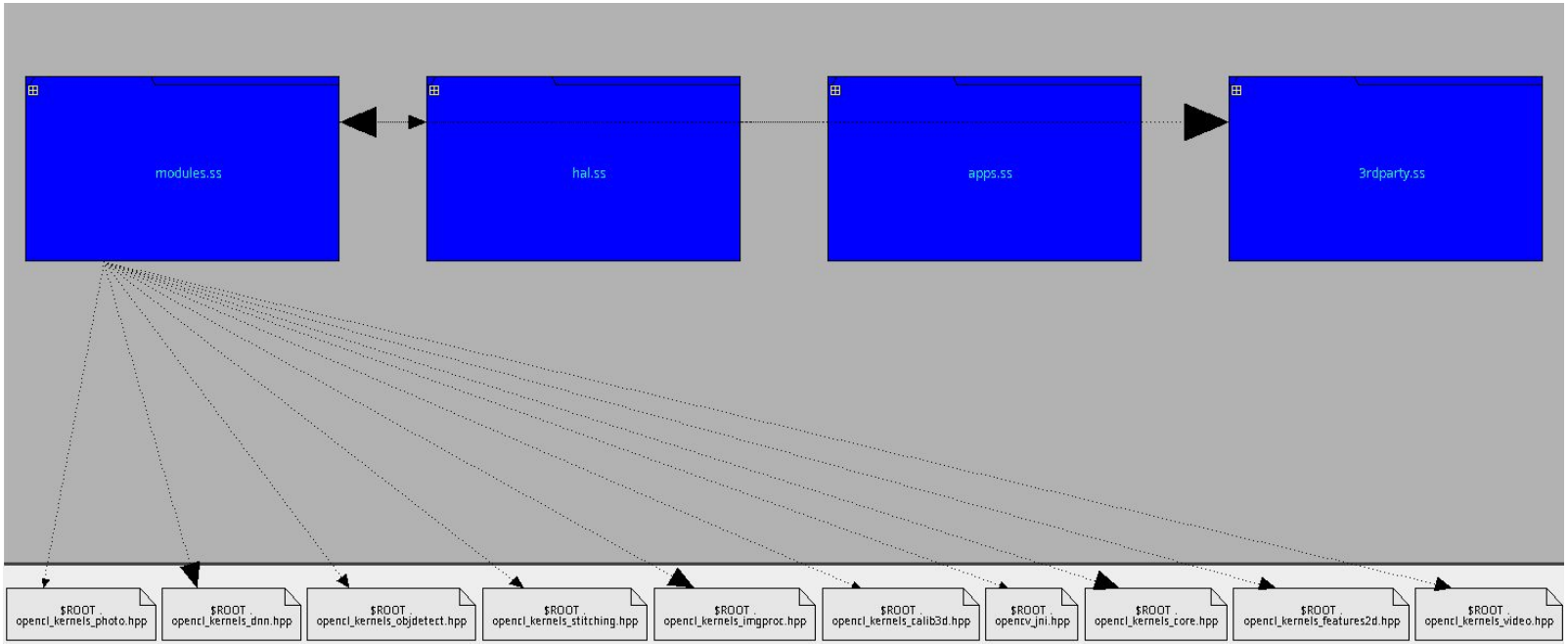


Figure 2: High level architecture of OpenCV captured from LSEdit.

- The module subsystem depends on the hardware acceleration layer (HAL).
- HAL provides an abstraction layer for module to delegate operations to hardware specific implementations without direct dependencies on hardware backends.

High-Level Concrete Architecture

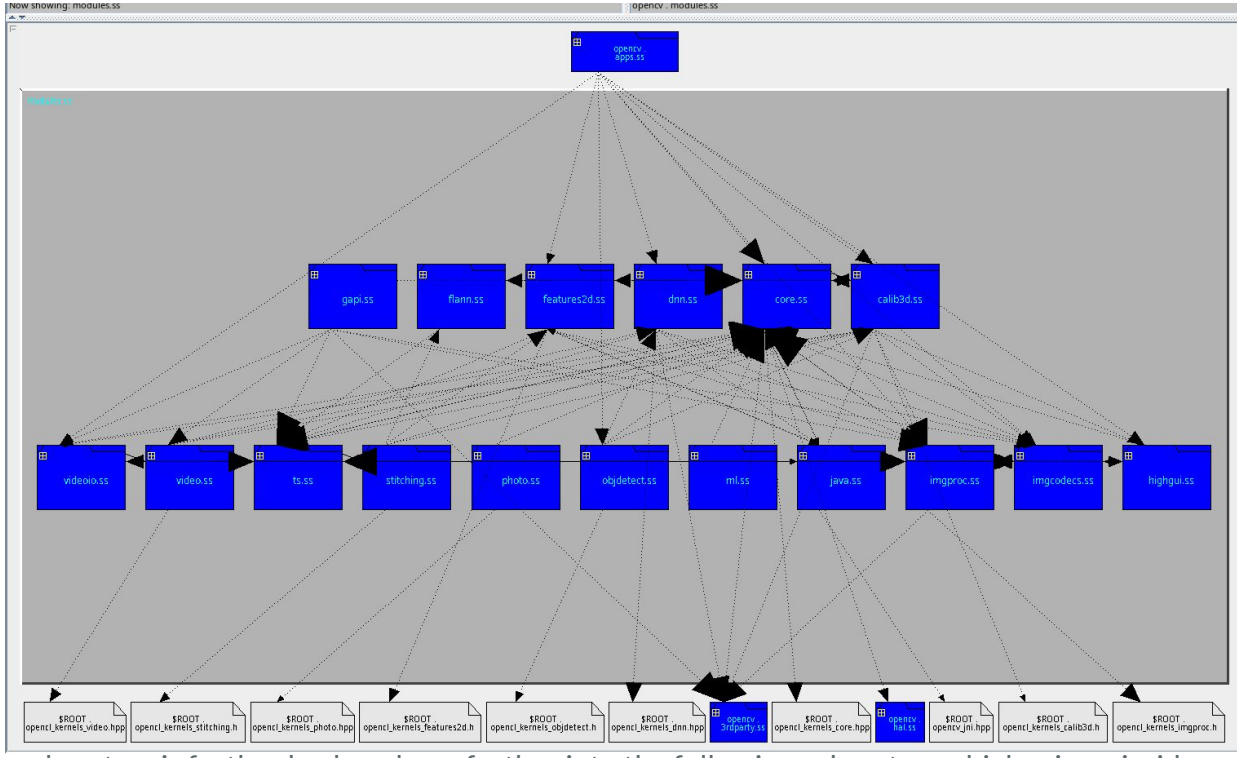


Figure 3:
High level architecture of "Module" captured from LSEdit.

- The module subsystem is further broken down further into the following subsystems: highgui, gapi, video, stitching, objdetect, dnn, ml, flann, calib3d, features2D, photo, videoio, imgcodecs and imgproc.

High-Level Concrete Architecture

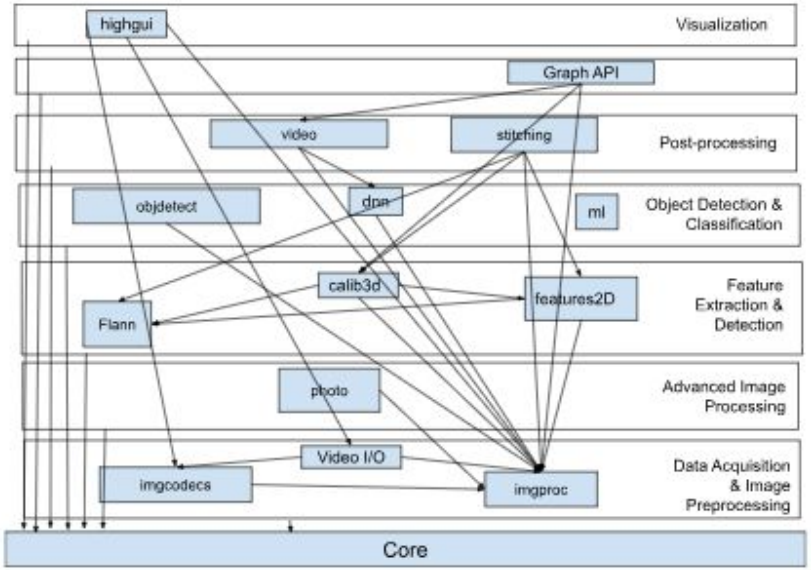
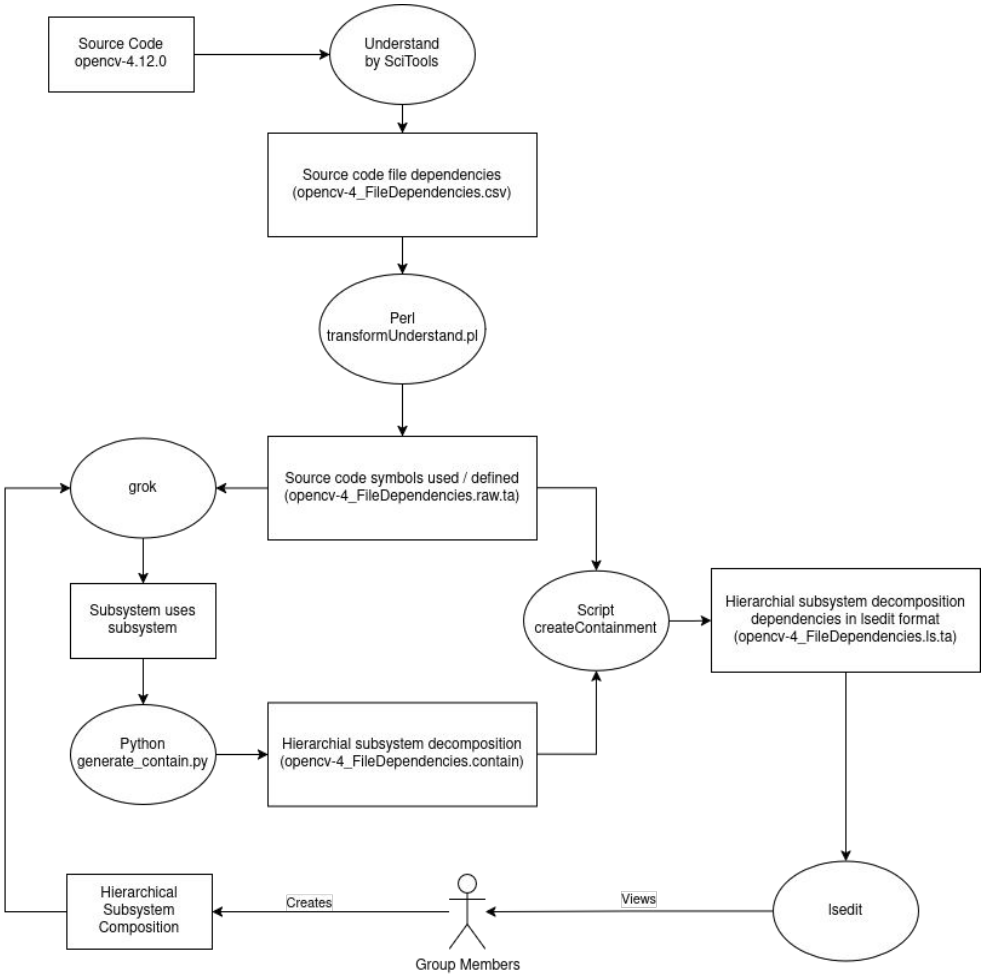


Figure 4: Derived Concrete Architecture

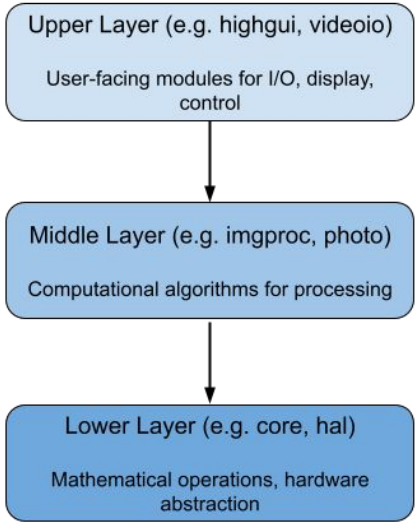
Derivation Process

- Input → OpenCV 4.12.0 source code
- Understand → Export .csv file dependencies
- Perl (transformUnderstand.pl) → Convert to .raw.ta
- Grok → Validate dependency format & structure
- Custom script (generate_contain.py) → Outputs .contain file
- createContainment → Merge → .ls.ta into LSEdit format
- LSEdit → Visualize subsystem graph
- Review (by group members)
- Refinement (with Grok)



Architectural Style

- Consists of Upper, Middle, and Lower Layers
- Upper Layer: Handles I/O, display, and user interaction [13]
- Middle Layer: Implements computational algorithms [13]
- Lower Layer: Provides mathematical operations and hardware abstraction [13]
- Enables easy maintenance and extensibility across the system



Architectural Design Patterns

- Facade Pattern: Simplifies API exposure
- Strategy Pattern: Runtime algorithm selection
- Factory Pattern: Object creation for platform-dependent implementations
- Template Method Pattern: Defines algorithm skeletons for processing steps

Architecture of imgproc

Modules [3]

- Image Filtering
- Geometric Image Transformations
- Miscellaneous Image Transformations
- Drawing Functions
- Color Space Conversions
- ColorMaps in OpenCV
- Planar Subdivision
- Histograms
- Structural Analysis and Shape Descriptors
- Motion Analysis and Object Tracking
- Feature Detection
- Object Detection
- Hardware Acceleration Layer

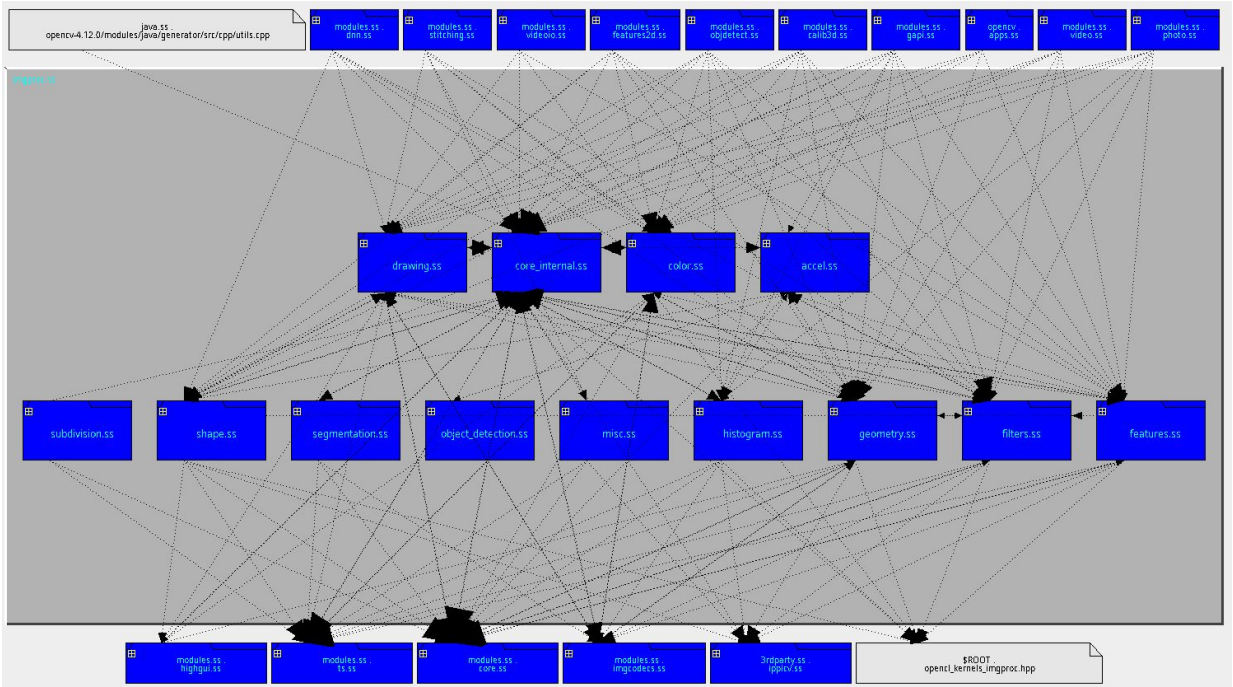


Figure 7: Architecture of imgproc captured from LSEdit.

Architecture of imgproc

Subsystems:

- Filters
- Geometry (Geometric image transformation)
- Colour_ops
  - Colour space conversions
  - Colour maps
- Morphology
- Segmentation
- Features
- Core_internals

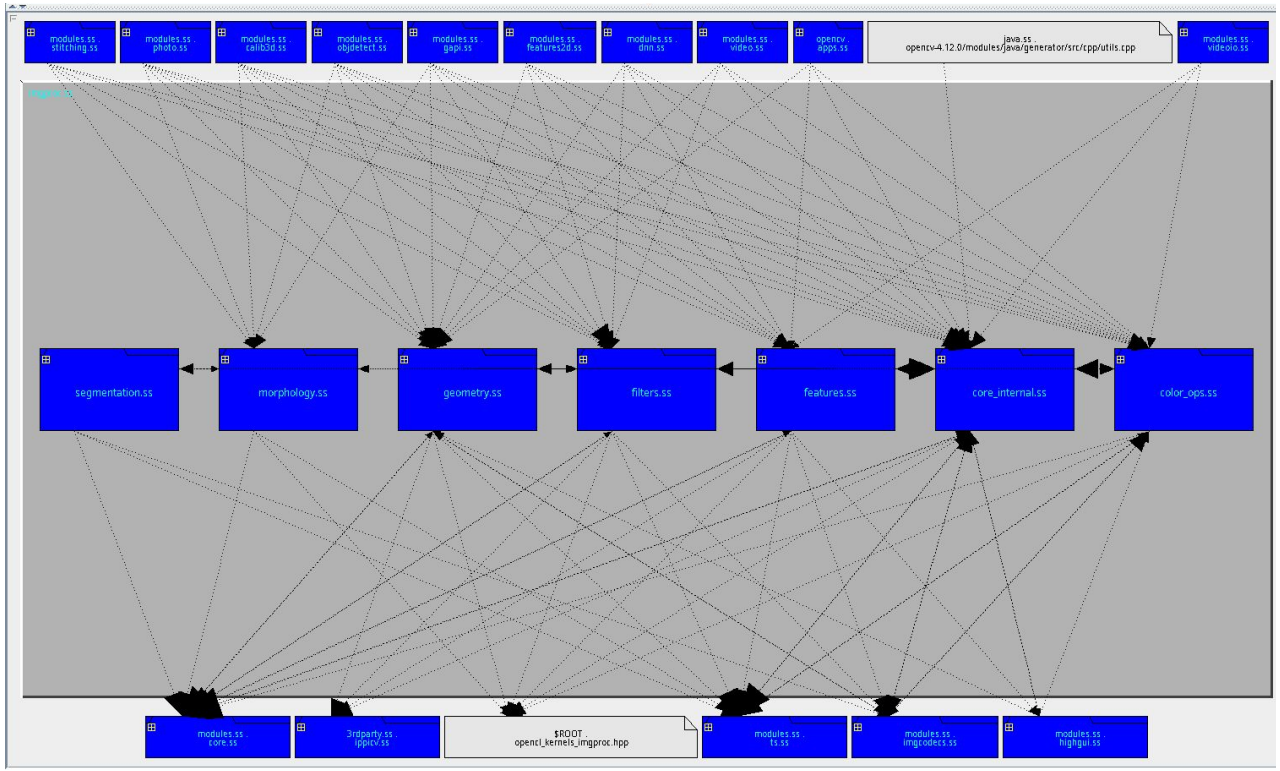


Figure 8: Architecture of imgproc captured from LSEdit using the listed subsystems.

Filter Subsystem

- 2 types of operations: linear & nonlinear [17]
- Both operations use neighbourhoods for filtering [17]
- Weighted sum used for linear operation [17]
- Morphological finds max and min value [17]
- Uses border extrapolation for more accurate results [17]
- Various functions to:
  - Smooth [17]
  - Detect edges [17]
  - Highlight features [17]
  - Reduce noise [17]

Geometric Image Transformations

- Change image shape/position without changing content [14]
- Deform pixel grid to form new layout [14]
- Output pixels trace back to original source [14]
- Reduces transformation errors [14]
- Preserves image details [14]

Morphology Subsystem

- Image processing operations that process images based on shapes.
- Typically performed on binary images (black and white)
- 2 inputs; the original image and the structuring element or kernel which determines the nature of the operation
  - Uses structuring element or kernel to probe and modify image
- 2 main morphological operators are Erosion and Dilation
  - Erosion involves the shrinking of the foreground objects
  - Dilation involves expanding foreground objects
- Other Operations:
  - Opening operation involves erosion followed by dilation to remove noise
  - Closing operation is dilation followed by erosion to close gaps
  - Morphological gradient provides the difference between dilation and erosion of an image
  - “Top Hat” is the difference between input image and the opening of the image
  - “Black Hat” is the difference between the closing of the input image and the input image [2]

Segmentation Subsystem

- Splits an image into meaningful regions/objects [15]
- grabCut separates foreground and background using a mask + rectangle [15]
- watershed grows regions from marked seeds [15]
- Pixels are assigned to regions or boundaries [15]
- Produces labeled components that follow object edges [15]

Core Subsystem

- Serves as foundation of OpenCV
- Provides functions used by other modules
- Imgproc depends on these utilities for: [12]
  - Matrix Operation
  - Mathematical calculations
  - Memory management
- Imgproc uses Core's cv::Mat for data storage and image manipulation [12]

Color Subsystem

- Dedicated to converting images between different color representations
- Performs color space conversion operations critical for image analysis
- Preprocessing
- Pattern Recognition
- Interacts with core, filter and geometry, and segmentation
- Color subsystem acts as a critical preprocessing step

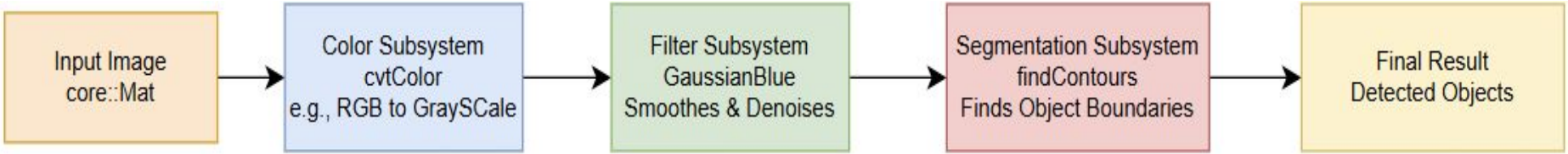


Figure 9: Interactions between systems

Features Subsystem

- Detect edges, line corners and descriptors [18]
- Uses Canny & Hough transformation techniques [18]
- Uses corner methods [18]
- Able to refine points [18]
- Explains keypoints [18]
- Enables object tracking and recognition [18]
- Can perform shape analysis [18]

Conceptual vs. Concrete Architecture -Reflexion analysis

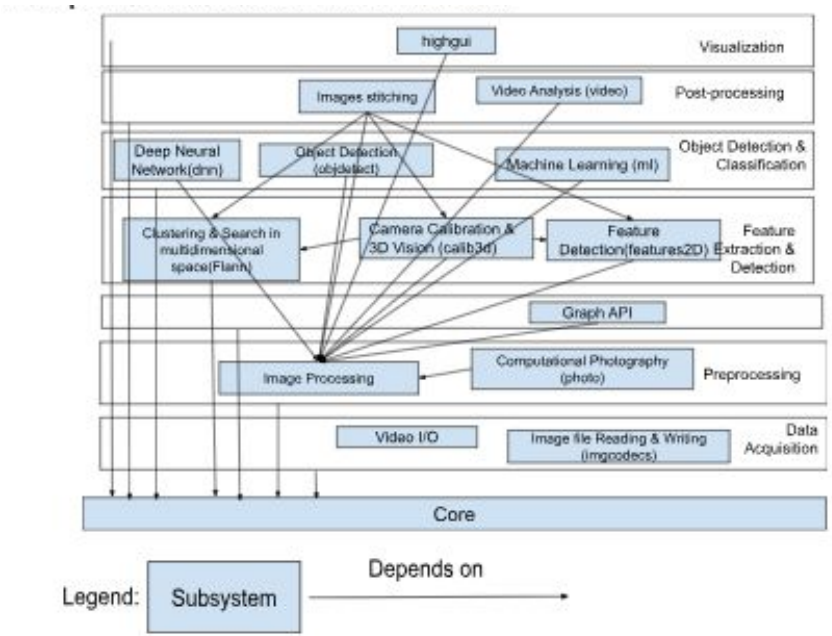


Figure 10. Conceptual Architecture

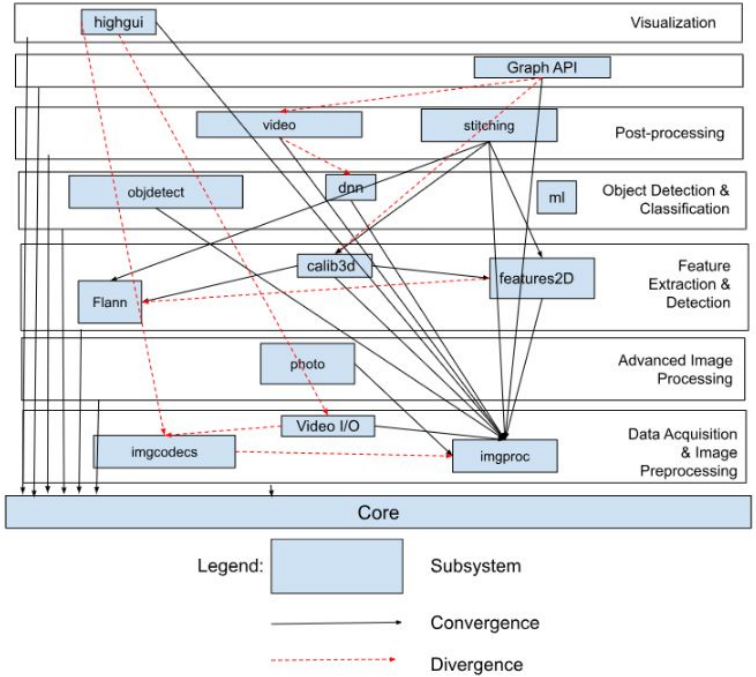


Figure 11. Reflexion Architecture

Dependency Investigation (Source StickyNotes)

Which?	The module imgcodecs depends on imgproc. Concrete Example From LSEdit: opencv-4.12.0/modules/imgcodecs/src/grfmt_gif.cpp cLinks opencv-4.12.0/modules/imgproc/src/color.cpp
Who?	Super [4]
When?	2024-12-07, part of the initial GIF decoder implementation.
Why?	The imgcodecs module has a required dependency on the imgcodecs module as shown in the build configuration file CMakeLists.txt; “ocv_add_module(imgcodecs opencv_imgproc WRAP java objc python)”. The concrete implementation example of this dependency: ● grfmt_git.cpp depends on on color.cpp as color conversion functionality is used in the GIF decoder. ● GIF decoders use cvtColor() from imgproc to convert decoded images between different color formats [4].

Dependency Investigation (Source StickyNotes)

Which?	The module videoio depends on imgcodecs. Concrete Example From LSEdit: opencv-4.12.0/modules/videoio/src/cap_images.cpp cLinks imgcodecs.ss ->opencv-4.12.0/modules/imgcodecs/src/loadsave.cpp et al.
Who?	Vadim Pisarevsky [5]
When?	2010-05-11
Why?	The videoio module has a required dependency on the imgcodecs module as it needs its functionalities for ● reading/writing individual frames in various image formats ● handling image format conversions during video encoding/decoding ● frame extraction ● video thumbnail generation This dependency is listed in the configuration file : “ocv_add_module(videoio opencv_imgproc opencv_imgcodecs WRAP java objc python)”. The implementation example of this dependency: ● The cap_images.cpp in videoio module implements image sequence capture/writing functionality, which requires the ability to read and write individual image files from disk. ● Calls imread() implemented in loadsave.cpp in imgcodecs module [5].

Dependency Investigation (Source StickyNotes)

Which?	G-API module has an optional dependency on the video module. Concrete Example From LSEdit: opencv-4.12.0/modules/gapi/test/cpu/gapi_ocv_stateful_kernel_test.cpp cLinks video.ss-> opencv-4.12.0/modules/video/src/bgfg_gaussmix2.cpp
Who?	AsyaPronina (Anastasiya Pronina) [6]
When?	2020-06-04
Why?	The dependency of G-API module on video module is optional and and it can function without the video module as show in the CMakeLists.txt: “ocv_add_module(gapi REQUIRED opencv_imgproc OPTIONAL opencv_video opencv_calib3d WRAP python)”. The implementation example is a test dependency: ● test file gapi_ocv_stateful_kernel_tests.cpp uses video module functionally(cv::BackgroundSubtractor) to test G-API's stateful kernel functionality. ● Stateful kernel testing framework was introduced in OpenCV 4.4 [6].

Dependency Investigation (Source StickyNotes)

Which?	G-API module has an optional dependency on the calib3d module. Concrete Example From LSEdit: opencv4.12.0/modules/gapi/src/backednds/cpu/gcpustero.cpp cLinks calib3d.ss ->opencv-4.12.0/modules/calib3d/src/stereobm.cpp
Who?	aDanPin [7]
When?	2021-03-31
Why?	The dependency of G-API module on calib3d module is optional and and it can function without the calib3d module as show in the CMakeLists.txt: “ocv_add_module(gapi REQUIRED opencv_imgproc OPTIONAL opencv_video opencv_calib3d WRAP python)”. The concrete implementation example: ● Dependency of the gcpustero.cpp on stereo matching algorithms like StereoBM implemented in calib3d. ● When calib3d is available, G-API can provide stereo vision capabilities and when it is unavailable, the stereo functionality is not included in the build [7].

Dependency Investigation (Source StickyNotes)

Which?	Video module has an optional dependency on the dnn module. Concrete Example From LSEdit: opencv-4.12.0/modules/video/src/tracking/tracker_vit.cpp cLinks dnn.ss->opencv-4.12.0 /modules/dnn/src/net.cpp
Who?	lpylpy0514 (PengyuLiu) [8]
When?	2023-09-19
Why?	The dependency of Video module on dnn module is optional and and it can function without the dnn module as show in the CMakeLists.txt: “ocv_add_module(video REQUIRED opencv_core opencv_imgproc OPTIONAL opencv_dnn WRAP python java)”. The video module uses dnn for deep learning-based tracking algorithms. An example is the dependency of the tracker_vit.cpp on dnn functionality (dnn:: Net class implemented in net.cpp) to load and run vision transformer neural network model for object tracking [8].

Dependency Investigation (Source StickyNotes)

Which?	The module features2d has an optional dependency on the flann module. Concrete Example From LSEdit: opencv-4.12.0/modules/features2d/src/matchers.cpp cLinks flann.ss->opencv-4.12.0 /modules/flann/src/miniflann.cpp
Who?	Multiple Contributions: -Vadim Pisarevsky: foundational FLANN wrapper code in miniflann.cpp [9] -Maria Dimashova: created the FlannBasedMatcher class in matchers.cpp [10]
When?	2011-07-13 (Vadim Pisarevsky) [9] 2010-10-29(Maria Dimashova)[10]
Why?	The dependency of features2d module on flann module is optional and and it can function without the video module as show in the CMakeLists.txt “ocv_define_module(features2d opencv_imgproc ${debug_modules} OPTIONAL opencv_flann WRAP java objc python js)”. Example: ● FlannBasedMatcher class in matchers.cpp depends on flann::Index class implemented in miniflann.cpp in flann module for fast approximate nearest neighbor search when matching feature descriptors [9],[10].

Dependency Investigation (Source StickyNotes)

Which?	The module highgui has an optional dependency on imgcodecs. Concrete Example From LSEdit: opencv-4.12.0/modules/highgui/include/opencv2/highgui.hpp cLinks imgcodecs.ss ->opencv-4.12.0/modules/imgcodecs/include/opencv2/imgcodecs.hpp
Who?	Alexander Alekhin [11]
When?	2016-08-25
Why?	The dependency of highgui module on imgcodecs module is optional and and it can function without the imgcodecs module as show in the CMakeLists.txt:"ocv_add_module(highgui opencv_imgproc OPTIONAL opencv_imgcodecs opencv_videoio WRAP python)". ● Part of OpenCV 3.0 modularization effort; original highgui module was split into three separate modules including imgcodecs, videoio, and highgui. This dependency was added to maintain backward compatibility from OpenCV 2.x where image I/O functions were part of the original highgui module. ● Dependency is also included to provide convenient access to image I/O functions (imread, imwrite) when users include the highgui header [11].

Dependency Investigation (Source StickyNotes)

Which?	The module highgui has an optional dependency on videoio. Concrete Example From LSEdit: opencv-4.12.0/modules/highgui/include/opencv2/highgui.hpp cLinks opencv-4.12.0/modules/videoio/include/opencv2/videoio.hpp
Who?	Alexander Alekhin [11]
When?	2016-08-25
Why?	The dependency of highgui module on videoio module is optional and and it can function without the videoio module as show in the CMakeLists.txt:"ocv_add_module(highgui opencv_imgproc OPTIONAL opencv_imgcodecs opencv_videoio WRAP python)". ● The highgui header conditionally includes videoio.hpp to provide convenient access to video I/O functions VideoCapture and VideoWriter when highgui is used. ● This maintains backward compatibility from OpenCV 2.x as these functions were part of the original highgui module[11].

Concurrency & Team Issues

- Concurrency:
  - Supports parallel image operations
  - Algorithms are thread-safe [1]
  - No shared mutable state
- Team Issues:
  - Distributed development model
  - Clear API boundaries
  - Consistent coding standards and pipelines
  - Reusability and maintainability

Sequence Case Example

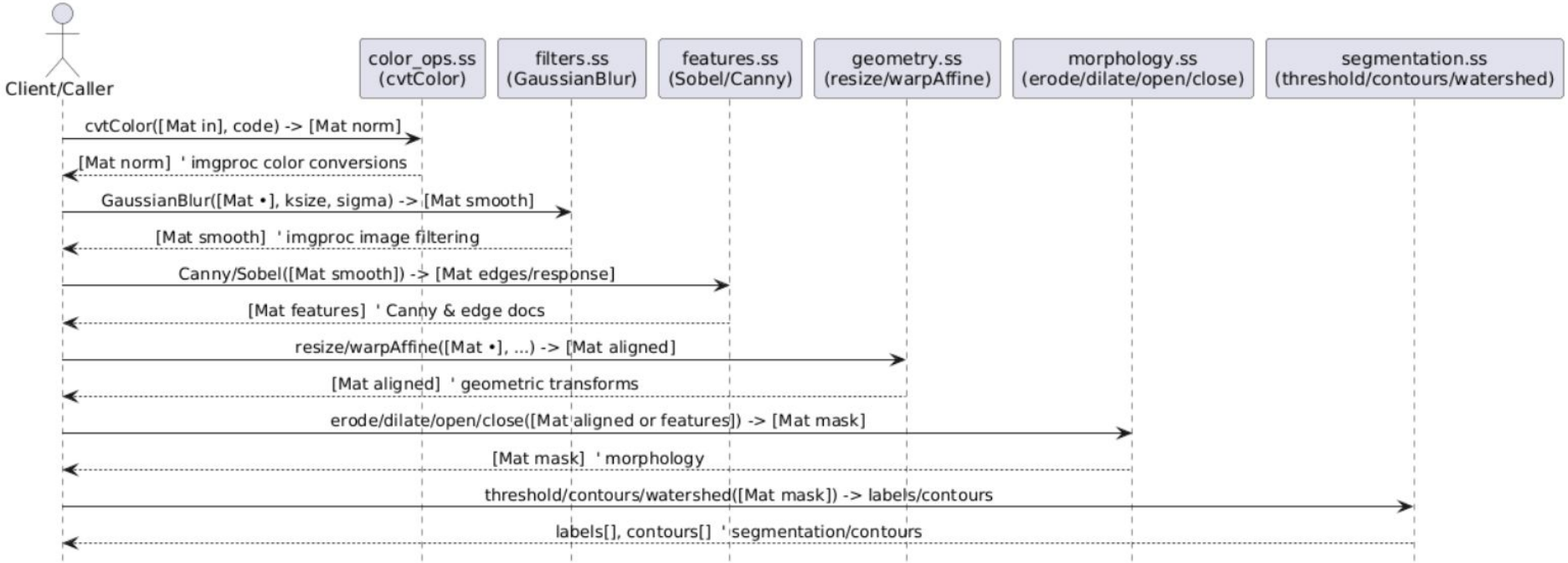


Figure 12 [16]. Sequence Diagram of Data and Control Flow Across imgproc Subsystems

Reflections & Lessons

- Understand and to recover large-scale software architectures
- Custom containment file improved understanding
- Importance of file when decomposing structure of subsystem
- Communication and teamwork was key
- Architectural pattern managed complexity
- Benefits of clear architecture and modular design

References

[1] “OpenCV: How to use the OpenCV parallel_for_ to parallelize your code,” *Opencv.org*, 2025. http://docs.opencv.org/4.x/d7/dff/tutorial_how_to_use_OpenCV_parallel_for_.html (accessed Oct. 28, 2025).

[2] “OpenCV: Morphological Transformations.” Accessed: Oct. 28, 2025. [Online]. Available: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

[3] “OpenCV: Image Processing.” Accessed: Oct. 06, 2025. [Online]. Available: https://docs.opencv.org/4.x/d7/dbd/group__imgproc.html

[4] opencv, “Merge pull request #25691 from redhecker:gifSupport · opencv/opencv@082cd7a,” GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/082cd7a74eab9a435c414e4fe7a5c0803d4d3b4d

[5] opencv, “‘atomic bomb’ commit. Reorganized OpenCV directory structure · opencv/opencv@127d664,” GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/127d6649a1c83397bf42799ac807af41aa507b30

[6] opencv, “Enable stateful kernels in G-API OCV Backend · opencv/opencv@b083c20,” GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/b083c20eb2eba7227f66dd10837609db9704fec6

[7] opencv, “Adding stereo operation and tests · opencv/opencv@e6f52b0,” GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/e6f52b0996d8e4f96ed13e501b4166805f09024b

[8] opencv, “Merge pull request #24201 from lpylpy0514:4.x · opencv/opencv@70d7e83,” GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/70d7e83dca7824e7e9d09e03630d1134628dc70a

[9] opencv, “upgraded to FLANN 1.6. Added miniflann interface, which is now used i… · opencv/opencv@562914e,” GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/562914e33bea921a0a235fbf2b6bba6830644e16

References P2

[10] “modified features2d interface; added algorithmic test for DescriptorM… · opencv/opencv@69e329c · GitHub.” Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/69e329c9

[11] opencv, “fix modules deps · opencv/opencv@063ac74,” GitHub. Accessed: Oct. 28, 2025. [Online]. Available: https://github.com/opencv/opencv/commit/063ac74369199c1cfcaefe266d963ea1c21db75d

[12] “Core functionality (core module),” OpenCV Documentation, accessed Oct. 28, 2025. [Online]. Available: https://docs.opencv.org/4.x/d0/de1/group__core.html

[13] OpenCV Documentation, https://docs.opencv.org/4.x/ (accessed Oct. 28, 2025)

[14] “Geometric image transformations,” OpenCV, https://docs.opencv.org/3.4/da/d54/group__imgproc__transform.html (accessed Oct. 28, 2025).

[15] “Image segmentation,” OpenCV, https://docs.opencv.org/4.x/d3/d47/group__imgproc__segmentation.html (accessed Oct. 28, 2025).

[16] “PlantUML,” PlantUML.com, https://plantuml.com/ (accessed Oct. 28, 2025).

[17] “Image Filtering,” OpenCV, https://docs.opencv.org/4.12.0/d4/d86/group__imgproc__filter.html (accessed Oct. 28, 2025).

[18] “Feature Detection,” OpenCV, https://docs.opencv.org/4.12.0/dd/d1a/group__imgproc__feature.html (accessed Oct. 28, 2025).